

SubScript Product: Features and Problems Solved

Updated protocol brief | June 2026

Prepared by Kristien

1. Introduction

The global economy has undergone a profound structural metamorphosis, transitioning from a model of discrete ownership to one of continuous access. While the "Subscription Economy" promises convenience and lower upfront costs, it has birthed a crisis of trust, transparency, and financial efficiency. Legacy banking infrastructure struggles to adapt to high-velocity digital commerce, creating fertile ground for predatory practices and technical failures. SubScript, the programmable payment layer for stablecoin commerce, enables one-time payments, recurring billing, usage-based charging, invoicing, and AI-native transactions through a Unified Payment Authorization (UPA) framework. Built natively on @Arc, SubScript migrates commercial logic from centralized databases to transparent, immutable smart contracts. SubScript chose @Arc as its foundational infrastructure due to its native USDC gas model, which eliminates the volatility of traditional networks, and its sub-second finality, which is essential for matching the performance of legacy payment processors while maintaining decentralized security.

1.1 Who We Serve

SubScript is a dual-market Product designed to bridge the gap between global consumers and modern enterprise.

For Consumers (B2C): SubScript provides a frictionless, fee-free subscription experience. It removes the stress of international card declines, eliminates hidden maintenance fees, and simplifies billing into a "set-and-forget" model that works globally without technical barriers.

For Businesses (B2B): SubScript delivers a comprehensive commercial billing infrastructure that handles far more than just recurring fees. It manages one-time B2B settlements and automated invoicing with sub-second financial settlement. By leveraging the UPA framework, it slashes processing overhead to a flat 1% and offers institutional-grade privacy via ArcaneVM for high-volume commercial operations.

2. Problems Solved: The Pathology of Legacy Subscriptions

The current digital commerce landscape is functionally broken, relying on a banking architecture that prioritizes merchant retention over consumer consent. SubScript is engineered to solve these systemic failures through its Unified Payment Authorization (UPA) framework, which ensures that one-time and subscription payments share the same infrastructure and execution rules:

2.1 Killing "Zombie" Subscriptions

Problem: In legacy systems, payments are "pulled" by merchants, often continuing long after a user attempts to cancel.

Solution: SubScript inverts this into a "programmable push" model where the user maintains absolute control via an onchain "Kill Switch" to instantaneously revoke permissions.

2.2 Eliminating Double-Billing

Problem: Technical database lags and asynchronous legacy systems frequently cause duplicate charges for a single billing cycle.

Solution: Smart contracts physically prevent this by enforcing strict billing intervals (e.g., one charge per 30 days) that cannot be bypassed by merchant errors.

2.3 Neutralizing Dark Patterns

Problem: Merchants often hide service fees in complex terms or implement undisclosed cancellation penalties.

Solution: Contract Transparency ensures all fees are hard-coded into the verifiable smart contract SLA, with UI warnings displayed before the user signs.

2.4 Ending Overdraft Penalties

Problem: Traditional "pull" billing can trigger predatory bank fees if an account has insufficient funds.

Solution: SubScript transactions are atomic; if funds are insufficient, the transaction simply fails without ever creating a negative balance or triggering fees.

2.5 Resolving Dispute Friction

Problem: Chargeback investigations are often "he-said, she-said" disputes based on private, opaque merchant records.

Solution: SubScript provides an indisputable record on the Arc ledger. Cryptographic verification serves as the single source of truth, eliminating costly arbitration.

2.6 The Reality of Dollar Cards vs. SubScript

In regions like Nigeria, obtaining a reliable dollar card to pay for global subscriptions is often a frustrating and expensive ordeal. When you compare that traditional system to SubScript, the difference in user experience is stark.

The Financial Drain (Traditional Cards): Providers typically charge an initial creation fee ranging from \$1 to \$5 for virtual cards. Traditional bank prepaid dollar cards require an issuance fee, such as N1,000. Users are often hit with monthly maintenance fees of around \$1 for virtual options. Traditional banks may charge an annual maintenance fee of \$10. Some providers charge flat transaction fees, like a \$0.90 penalty on all successful transactions. Failed transactions due to insufficient funds can trigger non-refundable penalty fees. The biggest hidden cost is currency conversion, where providers typically apply a markup of 1% to 3% on exchange rates.

The Friction (Traditional Cards): Getting a card requires heavy KYC verification, including submitting a BVN, uploading a valid ID, and taking a selfie. The verification process can take anywhere from a few minutes to 48 hours during high-traffic periods. Even after approval, cards frequently fail on global platforms due to billing address mismatches or strict restrictions on prepaid cards.

The SubScript

Solution: Zero User Costs: Setting up a SubScript wallet is completely free for the user. No Hidden Fees: There are zero creation fees, zero monthly maintenance charges, and zero transaction penalties for the subscriber. Instant Setup: Instead of waiting up to 48 hours for KYC approvals, a user simply clicks 'Continue with Google' for their wallet to be provisioned immediately. Banking-Independent Payments: Because SubScript operates natively with digital dollars (USDC), payments cannot be blocked by legacy card networks, arbitrary international banking restrictions, or regional firewalls. Transactions settle deterministically on-chain, provided the user has authorized the UPA payload and maintains sufficient wallet balances. Transparent Pricing: The subscriber pays exactly the advertised price of the service, while the merchant absorbs the flat 1% processing fee.

2.7 Structural Differentiation vs. Crypto Peers

Unlike streaming payment primitives (e.g., Superfluid) which require continuous, locked liquidity pools that collapse if a user's wallet balance dips below an active threshold, SubScript utilizes standard state allowances backed by Uniswap's Permit2. Funds remain completely static and liquid in the user's wallet until a decentralized keeper autonomously executes the exact billing cycle transaction, minimizing capital lockup and aligning perfectly with standard Web2 consumer behavior.

3. Features: What SubScript Does (The UPA Framework)

3.1 "Continue with Google" Setup

SubScript enables easy, secure onboarding for mainstream users through social login integration, removing the friction of seed phrase management. To ensure absolute non-custodial permanence, the initial onboarding flow mandates a secure, encrypted private key export phase, allowing users to retain full sovereignty and restore wallet access independently if their social authentication channel is ever compromised.

3.2 Set-and-Forget Automated Billing

The protocol reliably automates recurring payments via Account Abstraction, upholding service activity without manual monthly intervention.

3.3 Digital Dollar Receipts

Every transaction generates an instant, secure history on the ledger, providing users with a definitive, auditable record of their spending in USDC.

3.4 Zero-Fee Customer Experience

SubScript offers a frictionless experience where the user pays exactly the subscription cost-with zero hidden gas or network fees. This is possible because the protocol uses Circle's Gas Station and Paymaster smart contracts to sponsor network fees on behalf of the user. Because @Arc uses USDC natively for gas, these sponsorship costs are predictable and low, allowing them to be absorbed without impacting the user.

3.5 Fair Merchant Pricing

SubScript provides a transparent fee structure of 1% per successful transaction, significantly lower than the 2.9% + \$0.30 charged by legacy processors.

3.6 "Pay for Me" / Sponsored Subscriptions

Allows third parties like employers or parents to automatically cover a user's costs while maintaining user data privacy.

3.7 Fiat-to-USDC Onboarding

To ensure a frictionless experience, SubScript integrates direct fiat-to-USDC onramps. Users can fund their wallets via standard bank transfers; once the fiat is received, the system automatically swaps it for USDC and deposits it into the user's SubScript wallet. This eliminates the need for users to interact with external exchanges or understand complex bridging, making the transition from local currency to digital dollars instantaneous.

3.8 Permit2 Integration

Utilizes the Permit2 standard for efficient, programmable authorizations for bounded payment flows, replacing gas-heavy escrow contracts.

3.9 Absolute Statelessness

The router architecture holds zero balance across block boundaries, preventing protocol-drain attacks and maintaining network resilience.

3.10 Spam-Proof Communications

A "Proof-of-Transaction" financial firewall secures notifications and DMs, facilitating engagement for only legitimate participants.

3.11 Privacy Premium Tier (\$10/Month Baseline)

An institutional-grade tier leveraging ArcaneVM (Arc Privacy Sector). This environment enables confidential execution, allowing merchants to utilize trust domains and function-level access policies to selectively disclose billing data to authorized stakeholders. High-volume merchants utilizing this tier pay a flat operational fee of 10 USDC per month managed natively by the protocol, while standard transaction routing fees remain anchored to the core 1% ledger architecture.

3.12 DNS Registration

Merchants and users can register human-readable branded domain aliases as payment identities for easier identification.

3.13 Automated Notification Gateways

Managed enterprise-grade gateways handle high-volume commercial throughput and automated messaging across multiple channels.

3.14 Payment Links

To simplify merchant onboarding, SubScript provides a frictionless funnel through branded payment links (e.g., "subscript.xyz/pay/abc123"). These allow for instant, non-subscription based payments that benefit from the same security and transparency as the broader protocol.

3.15 Flexible Usage-Based Billing

For platforms without fixed payment plans, the SubScript Product leverages @Arc's low-latency finality to enable flexible, event-driven business models that replace static subscriptions with real-time usage billing. By shifting from time-based cycles to consumption-based settlements, platforms can implement:

API Token Consumption: Developers can bill users precisely for the number of API calls made or tokens processed by an AI model, rather than forcing them into a monthly tier that may overcharge for low usage or throttle them for high usage.

Dynamic Cloud Storage: Enterprise storage providers can charge fees calculated strictly by the gigabyte per day of storage used, upholding that users only pay for the capacity they actively occupy rather than paying for a pre-allocated block.

Pay-Per-View/Article Access: Content platforms can enable users to settle micropayments instantly for individual articles or video clips as they are consumed, eliminating the need for a monthly "all-access" pass that a casual reader might not fully utilize.

3.16 Invoice Engine

The automated Invoice Engine streamlines B2B operations by generating verifiable invoices with custom terms (e.g., "Due in 14 days"). These invoices enable payers to settle debts directly through the SubScript Product with full auditability.

3.17 Payment Execution Layer (Decentralized Keepers)

Recurring billing and automated UPA renewals require a highly reliable off-chain trigger. While early testing utilizes standard backend relayers, SubScript's production mainnet architecture actively transitions away from centralized CRON jobs to eliminate single points of failure. Instead, the Product scales via Chainlink Automation, an ultra-reliable and performant smart contract automation solution.

How Execution Happens: By using decentralized oracle infrastructure for automation, SubScript guarantees that contractual billing conditions are triggered instantly and deterministically. Automation nodes monitor the subscription parameters off-chain and broadcast the transaction when a billing cycle is due.

Gas Sponsorship: The execution gas consumed by these decentralized keepers is sponsored via Circle's Paymaster infrastructure. The cost is absorbed predictably in USDC, meaning the user never pays gas.

Failure Recovery: If a temporary RPC failure or extreme network congestion delays a keeper, the network continuously simulates the state off-chain and will automatically execute the rebill the moment conditions stabilize, facilitating that no billing cycles are skipped or lost.

3.18 Merchant Protection Layer (Programmable Commitments)

Legacy onchain protocols index heavily on user protection, often leaving merchants vulnerable to consumers who utilize a service and immediately revoke authorization. SubScript balances the scales by allowing merchants to enforce configurable terms directly within the UPA payload:

Service Lock Windows: Merchants can configure smart contracts to prevent users from revoking their Permit2 authorization immediately after consumption. The Product enforces a protocol ceiling of 72 hours for digital goods and 30 days for SaaS seats.

Minimum Commitment Periods: If a merchant offers a discounted rate in exchange for a 6-month commitment, the UPA enforces this duration, protecting the merchant's projected MRR.

Billing Grace Periods: If an authorization is valid but funds are low, the contract allows a programmable grace period, preserving the user's access while locking in the merchant's right to collect once the wallet is funded.

3.19 Smart Dunning Engine (Automated Revenue Recovery)

In the event a payment fails due to insufficient user balance, the transaction is not simply discarded. SubScript features a programmable dunning system designed to recover lost merchant revenue automatically without manual intervention.

Smart Retry Scheduling: The execution layer automatically queues rebill attempts on a configurable cadence (e.g., retrying on Day 1, Day 3, and Day 7).

Automated Communication: Every failed attempt triggers an off-chain webhook via the protocol's Notification Gateway, alerting the user via email or SMS to top up their wallet.

Automated Service Suspension: If the UPA authorization remains unfunded after the final retry schedule is exhausted, the smart contract automatically flags the subscription state as "Suspended," instantly notifying the merchant's backend to revoke service access.

4. Technical Pillars

Absolute Statelessness: The SubScript router architecture is designed to hold zero balance across block boundaries, preventing Product-drain attacks and maintaining network resilience.

Permit2 Integration: Depreciation of legacy escrow contracts in favor of Uniswap's Permit2 standard, providing programmable allowances while users maintain custody.

Network-Layer Exploitation: Bypassing brittle smart contract loops via @Arc v0.7.2 upgrade, utilizing native L1 transaction memos for auditability and native RPC batching to optimize throughput.

Spam-Proof Communications: An integrated notification system secured by a 'Proof-of-Transaction' financial firewall, which requires a valid, recent transaction to initiate communication, effectively neutralizing botnets.

ArcaneVM (Arc Privacy Sector): The core confidential execution environment. It utilizes default-deny contract isolation and an addTrustee function to enable "governed visibility"-allowing sensitive billing and payroll data to remain confidential on the public ledger while permitting selective disclosure to authorized payers, merchants, and the Product.

Gas Sponsorship & Predictability: SubScript utilizes Arc's stablecoin-native gas model and Circle's Paymaster infrastructure to sponsor network fees for users. This eliminates volatile 'gas' costs, upholding predictable,

USDC-denominated pricing for all transactions.

5. Legal and Compliance

SubScript is committed to operating within global regulatory frameworks. The Product integrates robust AML/KYC protocols for high-value B2B transactions and adheres to money transmission regulations in all active jurisdictions. By leveraging USDC, SubScript maintains a compliant posture regarding the movement of digital assets, ensuring that all commercial operations meet institutional standards for transparency and accountability.

Appendix: Quantum Resilience

@Arc is architected with a phased post-quantum resilience roadmap to protect against emerging threats.

1. Stopping Fund Theft (Quantum-Resistant Wallet Signatures)

The threat: Quantum computers could potentially forge standard signatures to authorize unauthorized transactions.

The solution: Arc implements beta support for post-quantum wallet signatures based on SLH-DSA-SHA2-128s, allowing the ecosystem to integrate signature verification that is mathematically resistant to quantum decryption.

2. Defeating 'Harvest-Now, Decrypt-Later' Attacks

The threat: Attackers may capture encrypted transaction data today, storing it to decrypt in the future when quantum hardware becomes accessible.

The solution: The Arc Privacy Sector (APS) utilizes post-quantum hybrid cryptography, combining classical algorithms (X25519) and post-quantum algorithms (ML-KEM-768) to protect the confidentiality of sensitive transaction details against future decryption.

3. Securing the Core Network & Validators

The threat: Quantum compromise of validator nodes or off-chain communication could jeopardize ledger integrity.

The solution: The roadmap includes upgrading node-to-node communication to TLS 1.3 with post-quantum hybrid key agreements, alongside adding post-quantum validator signatures to protect ledger integrity without compromising network speed.

Document Prepared By: Kristien

Date: June 2026